CSE 5017: Applied Machine Learning

Applied Machine Learning is the **disciplined use of machine learning** to solve real-world problems effectively, reliably, and responsibly. It is not just about choosing an algorithm or achieving high accuracy.

How do we apply machine learning properly to solve a real-world problem?

- Understand the real-world problem and decision context
- Determine whether ML is actually appropriate
- Formulate the problem as a valid learning task
- Build and evaluate models using sound experimental design
- Establish reliability, generalization, and trustworthiness
- Translate predictions into useful real-world decisions
- Deploy, monitor, and improve the solution over time

The Applied Machine Learning Pipeline

From a real-world problem to real-world impact

1. Frame the Problem

Problem understanding → ML suitability → task formulation → success criteria

2. Understand the Data

Data acquisition → provenance → EDA → data quality → labels → representativeness

3. Design a Valid Experiment

Data splitting → leakage prevention → baselines → preprocessing → evaluation protocol

4. Build and Evaluate Models

Feature engineering → model development → tuning → model selection → rigorous evaluation

5. Establish Reliability

Error analysis → uncertainty → calibration → robustness → generalization → interpretability → fairness

6. Operationalize and Improve

Application validation → deployment → MLOps → monitoring → feedback → retraining → impact assessment

Overall flow: Problem → Data → Experiment → Model → Evidence → Deployment → Impact

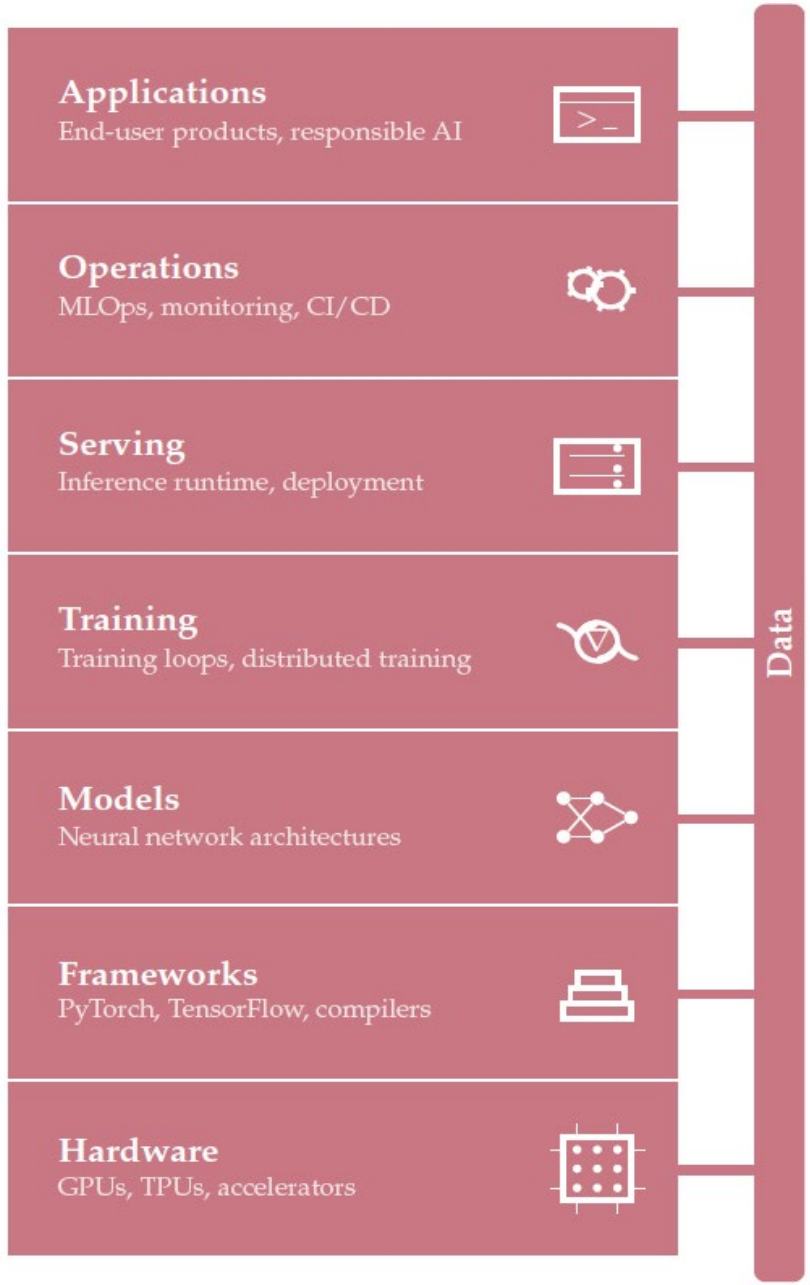
Applied ML is More Than Building an Accurate Model

Traditional view: Dataset → Algorithm → Accuracy
Applied ML view: Problem → Evidence → Decision → Impact

We must ask…	Why it matters
Are we solving the right problem?	A good model cannot fix a poorly defined problem.
Is the data appropriate?	Biased or unrepresentative data produces misleading results.
Is the experiment valid?	Leakage or inappropriate splits can create unrealistic performance.
Do we beat meaningful baselines?	Complexity must provide genuine value.
Where does the model fail?	Average performance can hide serious failure cases.
Will it generalize?	Real deployment data may differ from training data.
Do predictions improve decisions?	Better metrics do not automatically mean better outcomes.
Can we maintain it?	Models must remain reliable after deployment.

Applied Machine Learning

- from the mathematics that define a neural network to the hardware that executes it and the engineering decisions that make it run efficiently.
- For the past decade, ML research has been dominated by *model-centric* thinking focused on discovering new algorithms and architectures. That focus matters, but a model is not a system.
- A neural network that performs well in a research notebook is useless if it can not meet latency constraints, if its data pipeline can not handle the required volume, or if its energy cost exceeds its economic value.
- The gap between a working model and a production system is not merely an implementation gap; it is a gap in fundamental engineering principles.



What is a Machine Learning System?

Machine learning systems are software systems whose core behavior is determined by **parameters learned from data**, rather than only by explicitly programmed rules.

Three implications

- Performance depends jointly on **data quality, algorithm choice, and machine capacity**.
- Requirements are **end-to-end**: failure in data, model, or serving can change the user-visible result.
- Unlike traditional software, ML performance can change **even when code and model weights remain unchanged**, because the production data distribution may shift.

Common pitfall

> **The model is not the system.**

Data pipelines, feature transformations, serving infrastructure, monitoring, and feedback loops are also part of the ML system.

A high-performing model is useful only when the **complete system** works under real operating conditions.

The D·A·M Taxonomy

D — Data: What does the system learn from?

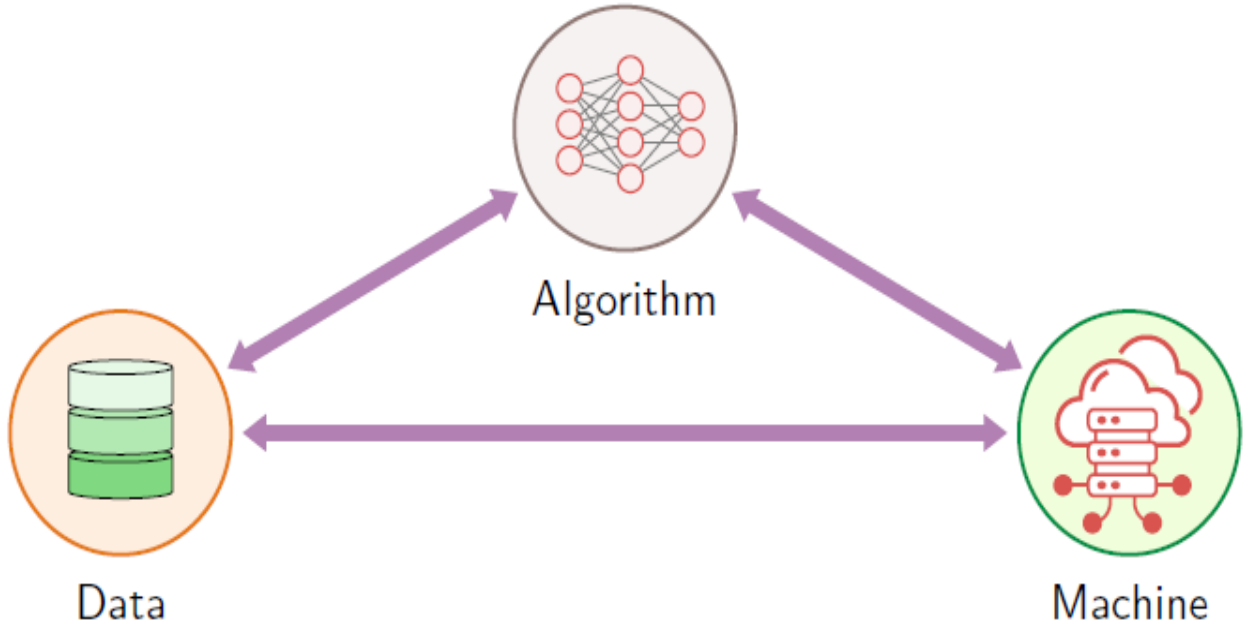
- Data coverage and representativeness
- Labels and data quality
- Distribution and dataset size

A — Algorithm: What does the system learn?

- Model structure and capacity
- Loss/objective
- Optimization and error patterns

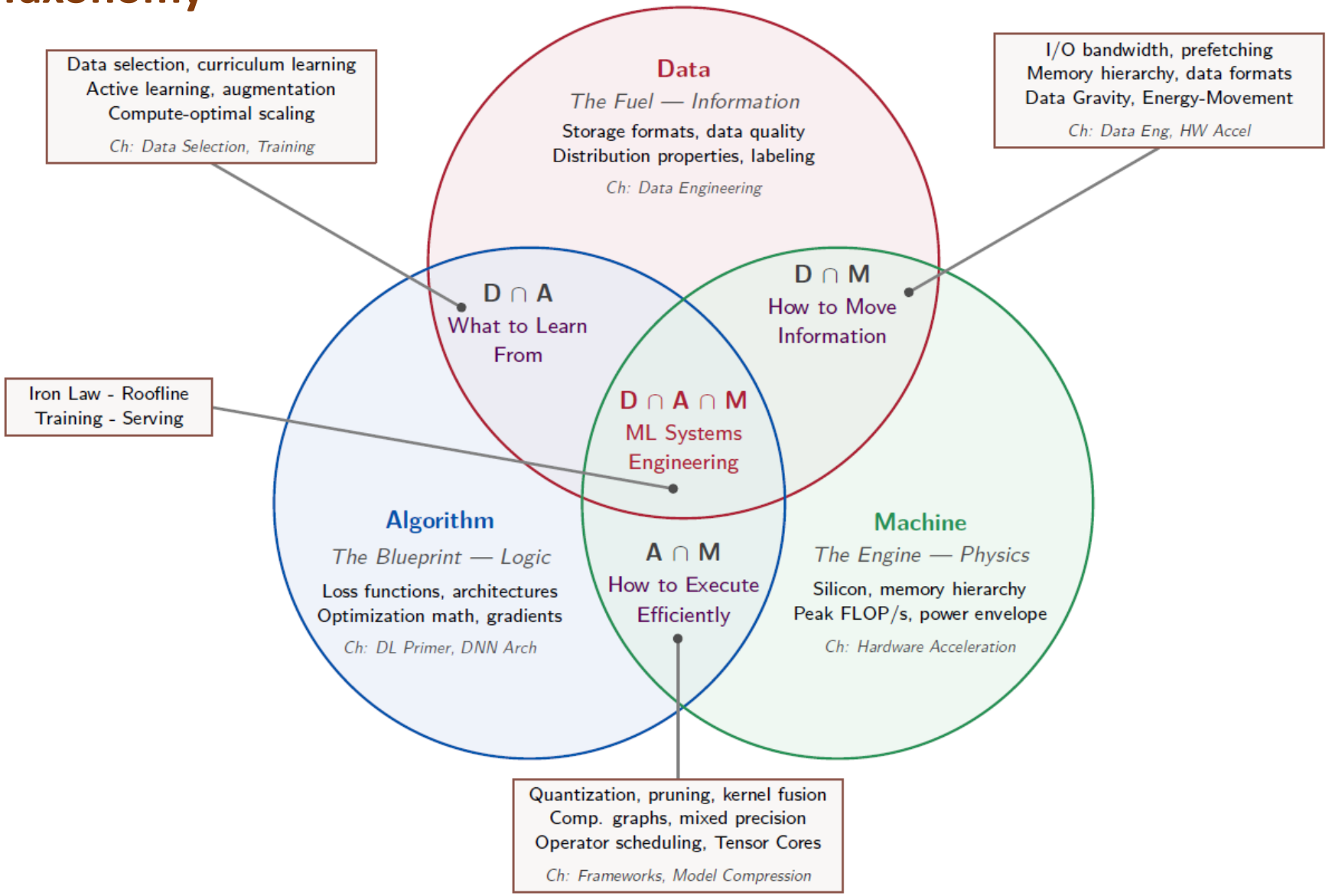
M — Machine: Can the system execute within its budget?

- Compute and memory
- Latency and throughput
- Bandwidth and utilization



changing one part can change the requirements imposed on the other two.

The D·A·M Taxonomy



The center — where Data, Algorithm, and Machine converge — is ML Systems Engineering itself.

The Phishing Detector That Passed the Benchmark but Failed in Production

Core lesson

A good model is not yet a good ML system.

What prevents the complete ML system from meeting its objective?

Production Requirements

The benchmark is only one part of the specification

Requirement	Target
Overall accuracy	≥ 95%
Phishing recall	≥ 90%
Per-email inference latency	≤ 30 ms
Peak traffic	No unacceptable delay

Offline evaluation

$$\text{Accuracy} = 96.5\%$$

$$\text{Phishing Recall} = 92\%$$

The model appears ready — but production adds latency.

The First Production Failure

A fast score on a benchmark is not the same as fast service in production

Observed during peak operation

38 ms

$38\ \text{ms} > 30\ \text{ms}$

Team proposal

“Buy a faster GPU.”

But do we know compute is the bottleneck?

Latency can come from data movement, workload size, hardware execution, runtime overhead, queueing, or orchestration.

The Iron Law of ML Systems

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{The Data Term}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{The Compute Term}} + \underbrace{L_{\text{lat}}}_{\text{The Latency Term}}$$

Symbol	Meaning	Unit
D_{vol}	Total data moved	Bytes
BW	Effective bandwidth	Bytes/s
O	Floating-point operation	FLOPs
R_{peak}	Peak compute rate	FLOP/s
η_{hw}	Hardware utilization	0–1
L_{lat}	Fixed latency/overhead	Seconds

Term	Expression	What it means	Typical causes
Data Movement	$\frac{D_{\text{vol}}}{BW}$	Time spent moving bytes	Weights, activations, inputs, memory/network transfers
Compute	$\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$	Time spent performing arithmetic	Matrix multiplication, convolution, attention, linear layers
Latency / Overhead	L_{lat}	Fixed system cost outside useful arithmetic	Scheduling, synchronization, kernel launches, networking, serialization

Total execution time is not determined by FLOPs alone, First identify **which term dominates**, then optimize.

The Iron Law

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{The Data Term}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{The Compute Term}} + \underbrace{L_{\text{lat}}}_{\text{The Latency Term}}$$

Training one batch of GPT-3 (175B parameters)

Term	Variables	Calculation	Time
Data	D_{vol}=350 GB, BW=2,039 GB/s	$350/2{,}039$	**0.17 s**
Compute	O=3.14×10^{14}, R_p=989 TF	$3.14\times10^{14}/(989\times10^{12}\times0.5)$	**0.64 s**
Overhead	L_{lat} (sync, sched.)	—	**0.05 s**
		$T_{\text{total}} \approx$	**0.86 s**

> **Diagnosis:** The compute term dominates for LLM training. Upgrading memory bandwidth alone yields diminishing returns.

The Iron Law

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{The Data Term}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{The Compute Term}} + \underbrace{L_{\text{lat}}}_{\text{The Latency Term}}$$

Diagnose the Bottleneck

A mobile inference pipeline has:

- $D_{\text{vol}} = 40$ MB, $BW = 100$ MB/s
- $O = 2 \times 10^9$ FLOPs, $R_{\text{peak}} = 20$ GF, $\eta = 0.6$
- $L_{\text{lat}} = 5$ ms

Data: $40/100 = 400$ ms

Compute: $\frac{2 \times 10^9}{12 \times 10^9} = 167$ ms

Overhead: 5 ms

$T_{\text{total}} = \mathbf{572\ ms}$

Data-bound!

Return on Compute (RoC)

$$\text{RoC} = \frac{\Delta\text{Accuracy}}{\Delta\text{Compute Cost}}$$

Approach	Accuracy Gain	Extra Cost	RoC
Large-scale pretraining	+5%	$100M	5×10^{-8} %/$
Fine-tuning pretrained model	+2%	$1K	2×10^{-3} %/$

$$\frac{2/1{,}000}{5/100{,}000{,}000} = \boxed{40{,}000\times}$$

$$\boxed{\text{Fine-tuning} \approx 40{,}000 \times \text{ higher RoC}}$$

- Scale can improve capability.
- RoC asks whether the marginal improvement justifies the marginal compute cost.

Compression Techniques

Distillation reduces the model needed to perform the task; quantization reduces the precision with which it does it.

Distillation vs Quantization

	Knowledge Distillation	Quantization
Core idea	Train a smaller **student** to imitate a larger **teacher**	Represent weights/activations using fewer bits
Example	12-layer teacher → 6-layer student	FP32 → FP16 → INT8 → INT4
Model structure	Usually becomes smaller	Usually remains structurally similar
Parameter count	Usually decreases	Usually unchanged
FLOPs	Usually decreases	Often similar operation count
Memory footprint	Decreases	Strongly decreases
Inference cost	Lower	Often lower
Accuracy risk	Student may lose capability	Lower precision may introduce error

Compression Techniques

Distillation reduces what the model needs to do; quantization reduces the precision with which it does it.

Technique	D_{vol}	O	Effective Compute	Main Benefit
Distillation	↓	↓	—	Smaller model, fewer operations
Quantization	↓↓	Often ≈	Can ↑	Less memory traffic + cheaper arithmetic
Distill + Quantize	↓↓↓	↓	Can ↑	Smaller model **and** lower precision

Suppose a teacher model requires:

$$O = 100 \text{ GFLOPs}, \qquad D_{\text{vol}} = 800MB$$

After distillation:

$$O = 40 \text{ GFLOPs}, \qquad D_{\text{vol}} = 350MB$$

After INT8 quantization:

$$D_{\text{vol}} \approx 100MB$$

So the optimization attacks **both major Iron-Law terms**:

$$\boxed{\text{less movement} + \text{less computation}}$$

Compression Techniques

Iron Law asks:

> Where is the system spending resources?

Compression asks:

> How can we reduce that resource requirement?

RoC asks:

> Is the resulting improvement worth the cost?

$$RoC = \frac{\Delta Accuracy}{\Delta Compute\ Cost}$$

Two Different Economic Questions

Situation	Appropriate Question
Pretraining / fine-tuning	**How much additional accuracy do we gain for additional compute?**
Distillation / quantization	**How much cost can we remove while preserving acceptable accuracy?**

Strategy	Accuracy Change	Resource Effect
Larger model	+1%	Compute ↑ substantially
Fine-tuning	+1%	Small additional training cost
Distillation	−0.5%	Model/FLOPs ↓ substantially
INT8 quantization	−0.3%	Memory + inference cost ↓ substantially
Distill + quantize	−0.8%	Potentially much smaller deployment cost

$$\text{Iron Law} \rightarrow \text{Identify bottleneck} \rightarrow \text{Distill / Quantize} \rightarrow \text{Remeasure} \rightarrow \text{Evaluate economic value}$$

Case Studies

Gopher (DeepMind, 2021)

- A large **decoder-only Transformer language model**
- Largest version: **280 billion parameters**
- Trained on about **300 billion tokens**
- Built to study what happens when language models become much larger

$$\sim 1.1 \text{ training tokens per parameter}$$

Main observation:

> Larger models generally performed better.

$$\boxed{\text{Bigger model} \rightarrow \text{Better performance}}$$

The model was very large, but it did not see enough training data for its size.

Case Studies

	Gopher	Chinchilla
Parameters	**280B**	**70B**
Training tokens	**~300B**	**~1.4T**
Model size	Very large	4× smaller
Training data	Less	~4.7× more

Despite being much smaller, **Chinchilla outperformed Gopher on many tasks** with roughly the same training-compute budget.

$$\text{Largest model} \neq \text{Best model}$$

$$\text{Balance model size} + \text{data} + \text{compute}$$

Final Takeaway

Do not begin with:

“Which model should I improve?”

Begin with:

“What prevents the complete system from meeting its objective?”

D·A·M → Measurement → Iron Law → Binding Constraint → Intervention → Remeasurement

A local improvement is useful only if it improves the end-to-end application.

Neural Network Architectures

Computation and Memory Requirements

Prerequisite: MLP, CNN, RNN, Transformer, and DLRM.



Objective 1
How much work must the machine do?
Operations • parallelism • critical path • arithmetic intensity

Objective2
How much data must the machine move?
Memory footprint • access pattern • reuse • data movement

Architecture as a work specification

Modeling view
Which function family is appropriate for the data?

Systems view
What does the architecture force the machine to compute and move?

Deployment view
Will the required latency, memory, energy, and cost be feasible?

Architecture determines the shape of the computation graph.

What the ML engineer sees

MLP	CNN	RNN	Transformer	DLRM
dense connectivity	locality + sharing	state + recurrence	all-to-all attention	sparse lookup

What the machine executes

Matrix multiplication	Convolution / sliding windows	Embedding lookup	Softmax + reductions	Gather / scatter

GEMM, MACs, and FLOPs

Operation

GEMM

General Matrix–Matrix Multiplication

Example: C = A × B

It tells us what operation the layer executes.

Work count

MACs

Multiply–Accumulate operations

One MAC ≈ a × b + c

It counts how many repeated multiply–add steps are needed.

Arithmetic count

FLOPs

Floating-point operations

1 MAC ≈ 2 FLOPs

It counts individual multiply and add operations.

Relationship

GEMM is the operation. MACs count the repeated multiply–accumulate work inside it. FLOPs count the individual arithmetic operations.

Use "FLOP" for an amount of work; use "FLOP/s" or "FLOPS" for a rate of hardware performance.

Counting Work in a GEMM

From matrix dimensions to MACs and FLOPs

Given:

A	×	B	=	C
M × K		K × N		M × N

Why MKN MACs?

- C has M×N output elements
- Each output element is a dot product of length K
- Therefore: M×N×K multiply–accumulate steps

Counting formula

MACs = MKN

FLOPs ≈ 2MKN

Example

$A_{100\times200} \times B_{200\times50} \rightarrow 100\times200\times50 = 1{,}000{,}000$ MACs ≈ 2,000,000 FLOPs = 2 MFLOPs

Note: exact addition counts differ slightly, but 1 MAC ≈ 2 FLOPs is the standard ML-systems approximation.

DLRM: Deep Learning Recommendation Model

ID → Embedding table → Dense vector → Feature interaction → Top MLP

Embedding table memory

Suppose an embedding table contains **100 million entries,** and each entry is represented by a **128-dimensional FP32 vector**.

Memory $= N \times d \times s$ $\quad 100M \times 128 \times 4 \approx \mathbf{51.2\ GB}$

If the table grows to **1 billion entries**: $1B \times 128 \times 4 \approx \mathbf{512\ GB}$

$$\text{Total Memory} = s \sum_i N_i d_i$$

The whole embedding table must be stored, even though only a tiny part is accessed at one time.

- **Notation**

N = number of embedding entries

d = embedding dimension

s = bytes per value

How much is accessed in one batch?

$$B = 2048, \qquad d = 128, \qquad s = 4 \text{ bytes}$$

Data accessed= B×d×s = 2048×128×4=1,048,576 bytes≈1 MB

If all 2,048 IDs are distinct: $\dfrac{2048}{100,000,000} \times 100 \approx \mathbf{0.002\%}$

$$\text{Embedding Payload} = B \times L \times d \times s$$

$$\text{Payload} = Bs \sum_i L_i d_i$$

L = average number of embedding rows retrieved for one recommendation example

Large-scale recommendation systems: workloads are often dominated by memory capacity and irregular access.

MLP: dense connectivity

Every output feature may depend on every input feature.

$$Y = X \cdot W + b$$

Main operation

Dense GEMM: $X[B \times d_{\text{in}}]$ multiplied by $W[d_{\text{in}} \times d_{\text{out}}]$.

Computation

MACs = $B \cdot d_{\text{in}} \cdot d_{\text{out}}$
FLOPs ≈ $2 \cdot B \cdot d_{\text{in}} \cdot d_{\text{out}}$
Excellent matrix-level parallelism.

Dominant memory

Weights: $d_{\text{in}} \cdot d_{\text{out}} \cdot s$

Input + output activations: $B(d_{\text{in}} + d_{\text{out}}) \cdot s$

Total ≈ $[d_{\text{in}} \cdot d_{\text{out}} + B(d_{\text{in}} + d_{\text{out}})] \cdot s$

Systems behaviour

Large batches reuse weights well. Small-batch inference may load many weights for little work, becoming memory-bandwidth sensitive.

Typical applications

Tabular data, structured prediction, regression/classification, and dense sublayers inside Transformers, DLRM, and many hybrid systems.

Notation

Y	output activations
X	input activations
W	weight matrix
b	bias vector
B	batch size
d_{in}	input features
d_{out}	output features
s	bytes per value
GEMM	general matrix multiply
MAC	one multiply + one add
FLOP	floating-point operation

MLP = simple dense math, but large layers require moving many weights.

CNN: local filters and weight sharing

A small filter is reused across many spatial positions.

Main operation

Convolution / sliding window. The same $K \times K$ filter is applied repeatedly over the image or feature map.

Intuition

CNNs assume locality: nearby pixels/features matter more. Reusing filters reduces parameters and increases arithmetic intensity.

$$\text{MACs} = B \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$$

$$\text{FLOPs} \approx 2 \cdot B \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$$

Dominant memory

Weights: $K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot s$
Input feature map: $B \cdot H_{\text{i}} \cdot W_{\text{i}} \cdot C_{\text{in}} \cdot s$
Output feature map: $B \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot C_{\text{out}} \cdot s$

Systems behaviour

Often compute-throughput bound because filter weights are reused. depth wise convolutions and small batches can become bandwidth/overhead sensitive.

Typical applications

Image classification, object detection, medical imaging, video, audio spectrograms, and spatial/sensor signals.

Notation

B	batch size
K	filter width/height
C_{in}	input channels
C_{out}	output channels
H_{i}, W_{i}	input height, width
H_{out}, W_{out}	output height, width
s	bytes per value

MAC	multiply + add
FLOP	floating-point op
Depthwise	one filter per channel

CNN = repeated local computation; weight reuse turns memory access into useful work.

RNN: sequential GEMM plus recurrent state

Each step depends on the previous hidden state.

$$h_t = f(x_t W_{\text{xh}} + h_{t-1} W_{\text{hh}} + b)$$

Main operation

Sequential GEMM + recurrent state. The computation is repeated for S time steps.

Computation

Vanilla RNN:
MACs ≈ $B \cdot S \cdot (d_{\text{in}}\ d_{\text{h}} + {d_{\text{h}}}^2)$
Dominant scaling often written as $O(S\ {d_{\text{h}}}^2)$.

Dominant memory

Parameters: $(d_{\text{in}}\ d_{\text{h}} + {d_{\text{h}}}^2) \cdot s$
Inference state: $B \cdot d_{\text{h}} \cdot s$
Training activations: $O(B \cdot S \cdot d_{\text{h}})$

Systems behaviour

The bottleneck is the sequential critical path: h_t must finish before h_{t+1}.
More compute units cannot fully remove this dependency.

Typical applications

Time series, streaming sensor data, speech/audio streams, sequential inference, and cases where compact state is useful.

Notation

h_t	hidden state at step t	f	activation (e.g., tanh)	d_{in}	input size	**GEMM**	general matrix multiply
x_t	input at step t	b	bias vector	d_{h}	hidden-state size	**MAC**	one multiply + one add
W_{xh}	input-to-hidden weights	S	sequence length (steps)	s	bytes per value		
W_{hh}	hidden-to-hidden weights	B	batch size	$O(\cdot)$	order of growth		

RNN = modest state memory, but step-by-step dependency limits parallelism.

Transformer: GEMM plus all-to-all attention

Parallel token processing is powerful, but dense attention grows with sequence length.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^{\mathrm{T}} / \sqrt{d_{\mathrm{k}}})\ V$$

Main operations

- Q/K/V/O projections: GEMM
- attention scores: QK^{T}
- softmax + reductions
- feed-forward network: GEMM

Computation per layer

Scaling-level view:
$O(B \cdot S^2 \cdot d + B \cdot S \cdot d^2)$
plus FFN: $O(B \cdot S \cdot d \cdot d_{\mathrm{ff}})$
Critical term: $O(S^2 d)$

Dominant memory

Parameters: $O(d^2 + d\ d_{\mathrm{ff}})$ values/block
Token activations: $O(B \cdot S \cdot d)$
Naïve attention scores: $O(B \cdot h \cdot S^2)$
Tiled exact attention may avoid storing full S^2 in HBM.

Autoregressive inference

KV cache for standard MHA:
$\approx 2 \cdot B \cdot S \cdot d \cdot s$ bytes per layer
Small-batch decoding can become memory-bandwidth bound because weights and KV state are repeatedly accessed.

Typical applications

LLMs, translation, document understanding, multimodal models, ViTs, and biological sequences.

Notation

Q, K, V	query, key, value	B	batch size	s	bytes per value
K^T	K transposed	S	sequence length	**MHA**	multi-head attention
d_{k}	key/query size	h	attention heads	**KV cache**	stored keys, values
d	model width	d_{ff}	FFN hidden width	**HBM**	GPU main memory

Transformer = excellent parallelism during full-sequence processing; attention and decoding introduce memory pressure.

Continuous Thought Machines

Luke Darlow[1] **Ciaran Regan**[1,2] **Sebastian Risi**[1,3] **Jeffrey Seely**[1] **Llion Jones**[1]
[1]Sakana AI, Tokyo, Japan
[2]University of Tsukuba, Japan
[3]IT University of Copenhagen, Denmark
{luke, ciaran, sebastianrisi, jeffrey, llion}@sakana.ai

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

Abstract

Biological brains demonstrate complex neural activity, where neural dynamics are critical to how brains process information. Most artificial neural networks ignore the complexity of individual neurons . We challenge that paradigm. By incorporating neuron-level processing and synchronization, we reintroduce neural timing as a foundational element. We present the Continuous Thought Machine (CTM), a model designed to leverage neural dynamics as its core representation. The CTM has two innovations: (1) **neuron-level temporal processing**, where each neuron uses unique weight parameters to process incoming histories; and (2) **neural synchronization as a latent representation**. The CTM aims to strike a balance between neuron abstractions and biological realism. It operates at a level of abstraction that effectively **captures essential temporal dynamics while remaining computationally tractable**. We demonstrate the CTM's performance and versatility across a range of tasks, including solving 2D mazes, ImageNet-1K classification, parity computation, and more. Beyond displaying rich internal representations and offering a natural avenue for interpretation owing to its internal process, the CTM is able to perform tasks that require complex sequential reasoning. The CTM can also leverage adaptive compute, where it can stop earlier for simpler tasks, or keep computing when faced with more challenging instances. The goal of this work is to share the CTM and its associated innovations, rather than pushing for new state-of-the-art results. To that end, we believe the CTM represents a significant step toward developing more biologically plausible and powerful artificial intelligence systems. We provide an accompanying **interactive online**